

Service Management & Field Workforce Platform

Research, website sections, layouts, architecture, MySQL database and folder structure

Built on PHP, HTML, CSS, JavaScript and MySQL. Based on the VibeSoft scope document (Customer + Admin + Technician platform). Date: 30 September 2026.

Section	What it covers
1. Technology stack	PHP, MySQL, HTML/CSS/JS and hosting needs
2. Login for all roles	One login page, role-based redirect
3. Deep research	Market, competition, scope gaps, timeline, compliance, risks
4. Website sections	About, before/after, checklist, process, why us, reviews, enquiry, FAQ, map
5. Layout patterns	Five modern home page layouts and options for each section
6. System architecture	Deployment, PHP layers, lifecycle
7. MySQL database	Relations, content tables, table list, SQL schema
8. Customer	Architecture, flow, folder structure
9. Admin	Architecture, modules, folder structure
10. Technician	Architecture, flow, folder structure
11. Shared code	Pieces used by all three roles
12. Feature matrix	Which feature is used by which role, and where it lives
13. Full folder structure	Every folder and file, in detail
14. Security and deployment	PHP-specific checklist
15. Next steps	What to decide before development starts

Note: the scope document does not name a technology stack. PHP and MySQL are the client's choice; the folder structure, class names and table designs are proposals. Market and pricing numbers come from vendor blogs and research summaries, so treat them as indicative. A live demo of four home page layouts is at <https://claude.ai/artifact/MmEbDi4JNc6bzY6BXGJbqW> (visible only to the account that created it until shared).

1. Technology stack

The platform is a server-rendered PHP application. Pages are PHP templates that output HTML and CSS, small JavaScript files add interactivity (sliders, accordions, photo upload, status buttons), and every record lives in a MySQL database.

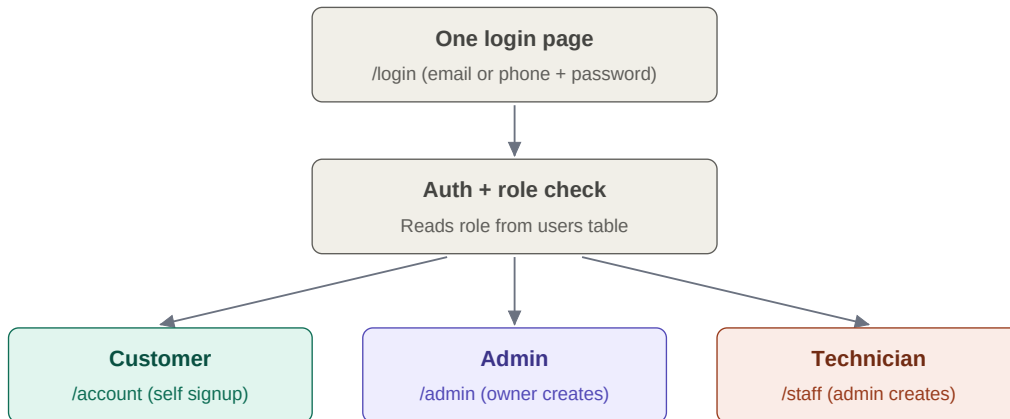
Layer	Choice	Notes
Language	PHP 8.2+	MVC structure with a single front controller (public/index.php)
Frontend	HTML5, CSS3, vanilla JavaScript	Mobile-first; CSS variables for a consistent theme
Database	MySQL 8 (or MariaDB 10.6+)	InnoDB tables, utf8mb4, foreign keys, accessed with PDO prepared statements
Charts	Chart.js (CDN)	Admin revenue and report charts
Email / SMS / WhatsApp	SMTP (PHPMailer) and provider APIs via cURL	Called through one Notifier class
Payments (phase 2)	Razorpay or similar	Not in the scope document
Hosting	cPanel shared host or VPS	Apache/Nginx, HTTPS, cron, PHP extensions: pdo_mysql, mbstring, fileinfo, gd, curl
Optional framework	Laravel	Adds routing, auth, validation and migrations out of the box; needs a slightly more capable host

Plain PHP or Laravel

Plain PHP with the structure in this document is light and cheap to host, but the team must write auth, routing, validation and CSRF protection itself. Laravel provides these and speeds up a 30-day build. If VibeSoft's team already works in plain PHP, use the structure here; if not, Laravel is the safer choice. The tables, screens and roles stay the same either way.

2. Login for all roles

Customer, Admin and Technician all use one **/login** page. There is no need for separate login pages. The system reads the **role** saved in the users table and sends the person to the right area.



Opening another role's page is blocked by the RequireRole middleware.

Role	How the account is created	After login	Access
Customer	Self signup (public)	/account	Own bookings, profile, invoices
Admin	First admin from seed.sql or a CLI script, later by the owner	/admin	Enquiries, staff, services, payments, reports, content, settings
Technician	Created by the admin (temporary password, changed at first login)	/staff	Only their own assigned jobs, earnings and history

Rules to follow

- Public signup creates customers only. Admin and staff roles must never be assignable from a public form.
- Check the role on the server in RequireRole.php (pages) and in every JSON endpoint. Hiding a menu item is not security.
- A technician can only read their own jobs. Enforce this in the SQL query (WHERE staff_id = current user).
- Regenerate the session id at login, hash passwords with password_hash, rate-limit login attempts.
- Phone OTP login suits Indian customers well. It is a suggestion; the scope document does not include it.

Login files

```
app/
|-- controllers/AuthController.php           # login, signup, logout, forgot/reset password
|-- models/User.php PasswordReset.php LoginAttempt.php
|-- middleware/
|   |-- RequireLogin.php
|   |-- RequireRole.php                     # customer / admin / staff
|   |-- VerifyCsrf.php
|   |-- RateLimit.php                       # login attempts
|-- core/
|   |-- Auth.php                           # session, password_hash / password_verify
|   |-- Csrf.php
|-- views/
|   |-- layouts/auth.php
|   |-- auth/
|       |-- login.php                       # common login for all roles
|       |-- signup.php                     # customers only
|       |-- forgot-password.php
|       |-- reset-password.php
|       |-- change-password.php            # staff first login
```

3. Deep research

Market context

- India's home services market was roughly Rs 5,100-5,210 bn in FY25. The organised online part is only Rs 41-43 bn (under 1%). Redseer expects the online part to grow 18-22% a year to Rs 85-88 bn by FY30.
- About 85-90% of online demand comes from the eight biggest cities, and providers are now expanding into Tier II and III cities. That leaves room for local players with their own site and dispatch system, although no source sized that demand.
- Urban Company only turned profitable in FY25 and margins are still thin, so marketplaces are hard businesses.

Competition

Tool	Aimed at	Indicative price
Jobber	Small and mid service businesses	Core plan about \$39/month
Housecall Pro	Small teams	Basic about \$59, Essentials about \$149/month on annual billing; extra users about \$35
ServiceTitan	Large companies	Quote only; reports suggest about \$245 per technician
Indian FSM tools (Zoho FSM, Zuper, FieldEZ, KaryaFlow and others)	Indian SMBs with GST workflows	One directory lists basic packages at Rs 600-8,000 per user per month

A custom build only beats a subscription when the client needs their own brand, a public booking site and full data ownership. For a 10-technician team, subscriptions could cost from under Rs 1 lakh to about Rs 10 lakh a year (my extrapolation). Show the client this build-versus-buy comparison before signing.

Gaps in the scope document

- **Online payment:** only amount and status are recorded. There is no payment gateway or payment link.
- **Invoice:** 'basic invoice/receipt information' only. A GST-compliant invoice should be specified.
- **Notifications:** the channel (SMS, email, WhatsApp, push) is not defined.
- **Mobile use:** web only. Technicians need a mobile-friendly or PWA flow, photo upload and weak-network behaviour.
- **Location:** no map, service-area check or live tracking.
- **Workflow:** 'Acceptance' is not detailed; there is no review module; cancel, reschedule, refund and dispute are missing.
- **Commercial terms:** hosting, domain, SMS/WhatsApp costs, source-code ownership, warranty and support are not stated.
- **Website content:** About, FAQ, before/after gallery, reviews, map, process and checklist are not in the scope (see section 4).
- **Language:** Hindi support is not mentioned, although Tier-2/3 audiences may need it.

Timeline and cost

- Thirty days for three role-based interfaces, a booking engine, reports and QA is aggressive. It is achievable as an MVP on a framework or template with tight scope control. Week 3 is the riskiest week.
- Cost benchmarks vary widely: one Indian guide gives Rs 1.5 lakh for a simple MVP up to Rs 1.5 crore for enterprise; another gives \$8,000-15,000 for a lean MVP and \$18,000-45,000 for mid-level.
- Budget for running costs: support is typically 15-20% of build cost per year, and cloud hosting can run Rs 5,000-50,000+ a month. A PHP and MySQL stack can start much cheaper on shared hosting.

Compliance (India)

- **DPDP Act:** full obligations apply from 13 May 2027. There is no blanket exemption for small businesses and penalties can reach Rs 250 crore. Build consent capture, privacy notices, deletion and audit logs from the start.

Sources differ slightly on dates, so verify with a lawyer.

- **Gig-worker law:** applies when technicians are independent contractors engaged through a platform that counts as an aggregator. The Social Security Code has been in force since 21 November 2025 and central rules were notified on 8 May 2026. Aggregators contribute 1-2% of turnover, capped at 5% of worker payouts, and register workers within 45 days. Rajasthan and Karnataka have state laws too. If technicians are employees, normal labour rules apply. Get legal advice.
- **WhatsApp:** a simple chat link is free. Automated messages use the paid API: Meta's India rates are Rs 0.8631 per marketing message and Rs 0.115 per utility message, plus 18% GST and a provider fee. From 1 October 2026 service messages are billed too.

Business risks

- **Platform leakage:** in a marketplace of independent technicians, customers and technicians can deal directly. Homejoy struggled with this. If technicians are the client's own employees the risk is low.
- **Adoption:** if technicians skip status updates and photos, the data goes stale.
- **Cold start:** a booking site with no traffic stays empty. It needs SEO, local ads or an existing customer base.
- **Scope creep** against a fixed 30 days, and **vendor lock-in** if IP, hosting and support terms are unclear.

Recommendations

- Clients: get the price, tech stack, hosting ownership, source-code rights and a support agreement in writing, and compare 2-3 year cost against Zoho FSM or Zuper.
- Ask for a phase-2 list: payment gateway, GST invoice, WhatsApp automation, PWA/offline mode, DPDP consent.
- Vendors: add a 'not included / optional add-ons' section, done-criteria per module, the assumptions behind the 30 days, and a compliance section.
- Decide first: is this a tool for one business or a product for many, and are technicians employees or contractors?

4. Website sections needed

The client wants these customer-facing sections: About, before/after images, checklist, process, why choose us, reviews, enquiry form, FAQ and a location map. Two reference sites were reviewed: primodomus.com and the Qleen demo on bold-themes.com. Neither homepage showed a before/after gallery, so that one is new work.

Section	Primodomus	Qleen demo	What we build
About	Short intro, read-more link, stat counters	Four values and a satisfaction counter	About block on the home page plus a full /about page: story, values, stats
Before/after	Not on the homepage	Portfolio pages in the menu	Draggable slider gallery, filterable by service, managed by admin
Checklist	Inclusions appear inside FAQ answers	'What is included' lists per service and package	Included / not-included list per service. Could double as a technician tick-list at completion (suggestion)
Process	Steps only inside an FAQ answer	Clear 3-step booking process	4 steps mirroring our workflow: Book, Confirm and assign, Visit, Done and invoice
Why choose us	Three cards plus counters	Trust badges: background-checked, insured, no contracts	4-6 icon cards, editable. Claim only what is true
Reviews	Only a rating stat on the page seen	Testimonial slider with photo, name, rating	Review slider and rating summary from real completed jobs, with admin approval
Enquiry form	Location picker, then booking; phone OTP login	Request Service button, quote page, price estimator	Our booking form plus a short home-page enquiry; price estimator optional
FAQ	Accordion with six questions	Accordion component	Accordion with global and per-service questions, admin-editable
Location map	'Use my location' or pick a city; 'coming soon' for unserved areas	Address in footer; map component	Map on the contact page, service-area list, pincode check on the booking form

Extras worth adding

- A sticky mobile bar with Call, WhatsApp and Book (Primodomus keeps Home, Booking and WhatsApp fixed at the bottom).
- Service packages in tiers, as Qleen does, if the client sells them.
- Legal pages: Terms, Privacy and Refund. They are needed for data-protection compliance anyway.
- Phone OTP login for customers.

Suggested home page order

1. Hero with quick enquiry
2. Service categories
3. Why choose us with stat counters
4. About
5. Process
6. Checklist or packages
7. Before/after gallery
8. Reviews
9. FAQ
10. Enquiry form and map
11. Footer with contact and legal links, plus the sticky call/WhatsApp bar

Impact on the project

- **New MySQL tables:** faqs, gallery_items, service_checklist_items, process_steps, service_areas, settings (see section 7). reviews gets an is_approved flag.

- **New admin module, 'Website content':** FAQs, gallery, checklists, review approval, service areas, About text and stats.
- **New customer pages and partials:** about, faq, gallery, plus before-after, checklist, process, why-choose-us, reviews, faq-accordion, map and enquiry-form partials.
- **Scope and timeline:** the original document lists hero, categories, highlights, CTA and contact only. The rest should be written into the quote as separate items. To protect the 30 days, keep About, process and why-choose-us as fixed template content at first; make only FAQs, gallery, checklists and reviews admin-editable in phase 1.

Cautions

- Qleen's testimonials and statistics are demo content. Use only real reviews and numbers from the client.
- A Google Maps embed is easy, but an interactive map may need an API key and billing. OpenStreetMap is a free alternative.

5. Layout patterns

Five modern home page layouts. All are built from the same sections; they differ in the hero and in how services and proof are arranged. A working demo with four of them is at <https://claude.ai/artifact/MmEbDi4JNc6bzY6BXGJbqW>.

A. Split hero with floating enquiry card

Headline, image and quick enquiry all sit above the fold. Best for one business that wants leads.

This wireframe illustrates a 'Split hero with floating enquiry card' layout. The top section is divided into two main areas. On the left, there is a headline area represented by three horizontal bars of varying lengths. Below the headline, there are two buttons: 'Book now' (dark) and 'Call us' (light). On the right, there is a large rectangular area labeled 'Hero image'. Overlaid on the bottom of the hero image is a floating enquiry card. This card contains four input fields labeled 'Name', 'Phone', 'Service', and 'Area', followed by a 'Get quote' button. Below the enquiry card, there are four service category buttons: 'AC service', 'Electrician', 'Plumber', and 'Cleaning'.

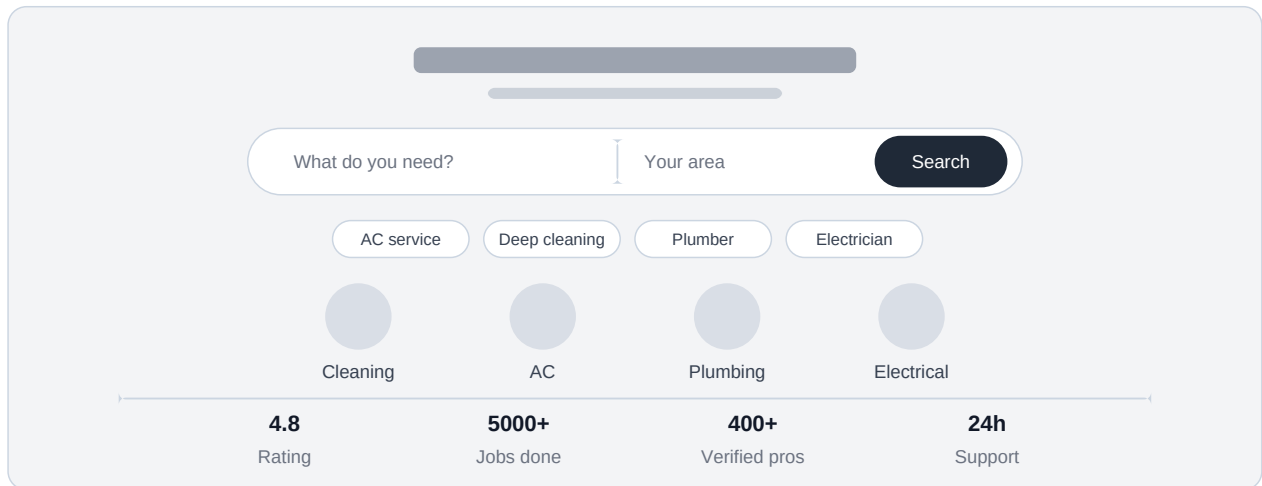
B. Bento grid

Services, a before/after preview, rating and stats share one block grid. Modern, and packs a lot of proof into little space.

This wireframe shows a 'Bento grid' layout. The grid is composed of several rectangular blocks of different sizes. The top-left block is a large blue rectangle with the text 'Home cleaning' and 'From price + Book now'. To its right is a light gray rectangle labeled 'Before / after'. Below the blue block is a white rectangle with '4.8 rating'. To the right of the 'Before / after' block are two stacked white rectangles labeled 'AC repair' and 'Plumbing'. Below the '4.8 rating' block is a wide white rectangle with the text 'Why choose us: verified, insured, on time'. To the right of this wide block is a white rectangle labeled '1000+ jobs'.

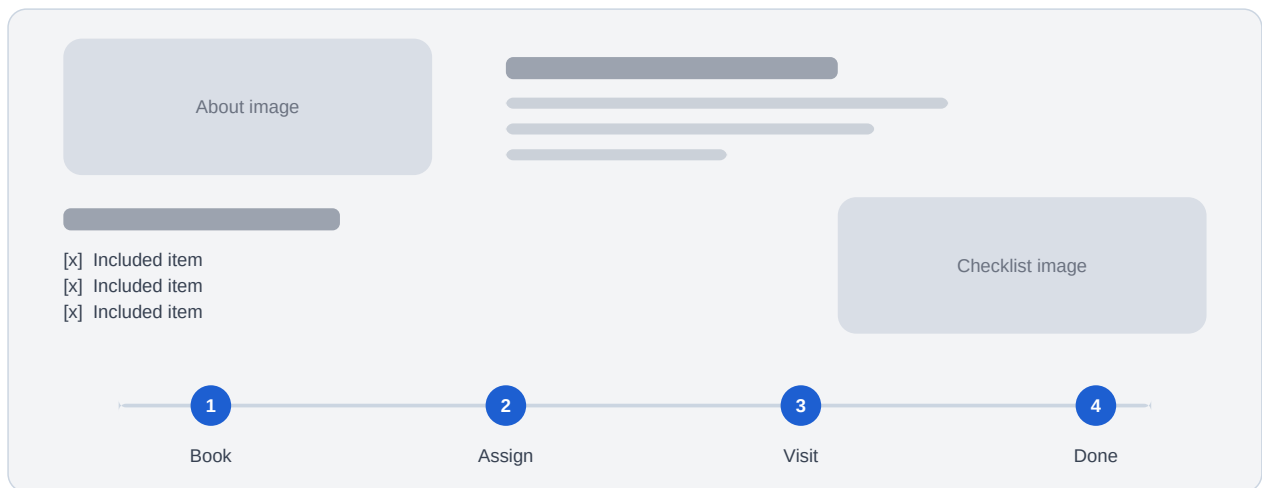
C. Search-first hero (marketplace style)

Large search for service and area, quick chips, category circles and a stats strip. Best for many different services.



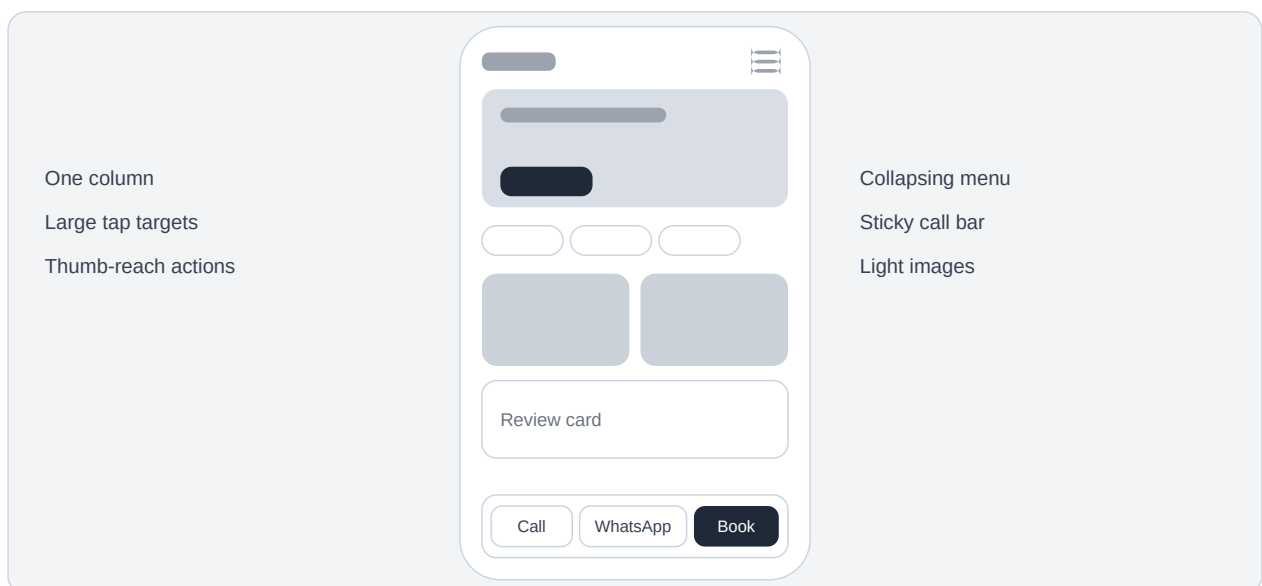
D. Zig-zag storytelling

Image and text alternate for About and checklist, with a numbered process strip. Easy to read and simple to build.



E. Mobile-first stack with sticky call bar

One column of cards with Call, WhatsApp and Book always in reach. Most visitors to a local service site are on phones, so every layout should collapse into this.



Layout options for the other sections

Section	Pattern options
Before/after	Draggable slider on one image; filterable cards by service; masonry grid that opens a lightbox
Reviews	Card carousel with a rating summary; masonry wall; one large quote with avatar thumbnails
FAQ	Simple accordion; two columns with a sticky heading; category tabs with a search box
Location map	Map beside an address card; full-width map with a floating info card; service-area chips with a pincode checker
Why choose us	Three icon cards; horizontal icon strip with counters; alternating rows
Checklist	Two columns (included / not included); tabs by package; expandable groups by area
Process	Horizontal numbered timeline; vertical timeline on mobile; cards joined by connector lines
Enquiry form	Floating card in the hero; section next to the map; slide-up drawer on mobile

Style rules for a modern look

- One accent colour plus neutral greys.
- Rounded cards (12-16px) with thin borders and no heavy shadows.
- Large headings with plenty of white space.
- Real photos of the client's own work. Stock images make a service site look generic.
- The same spacing and button style on every page.

Suggested combination

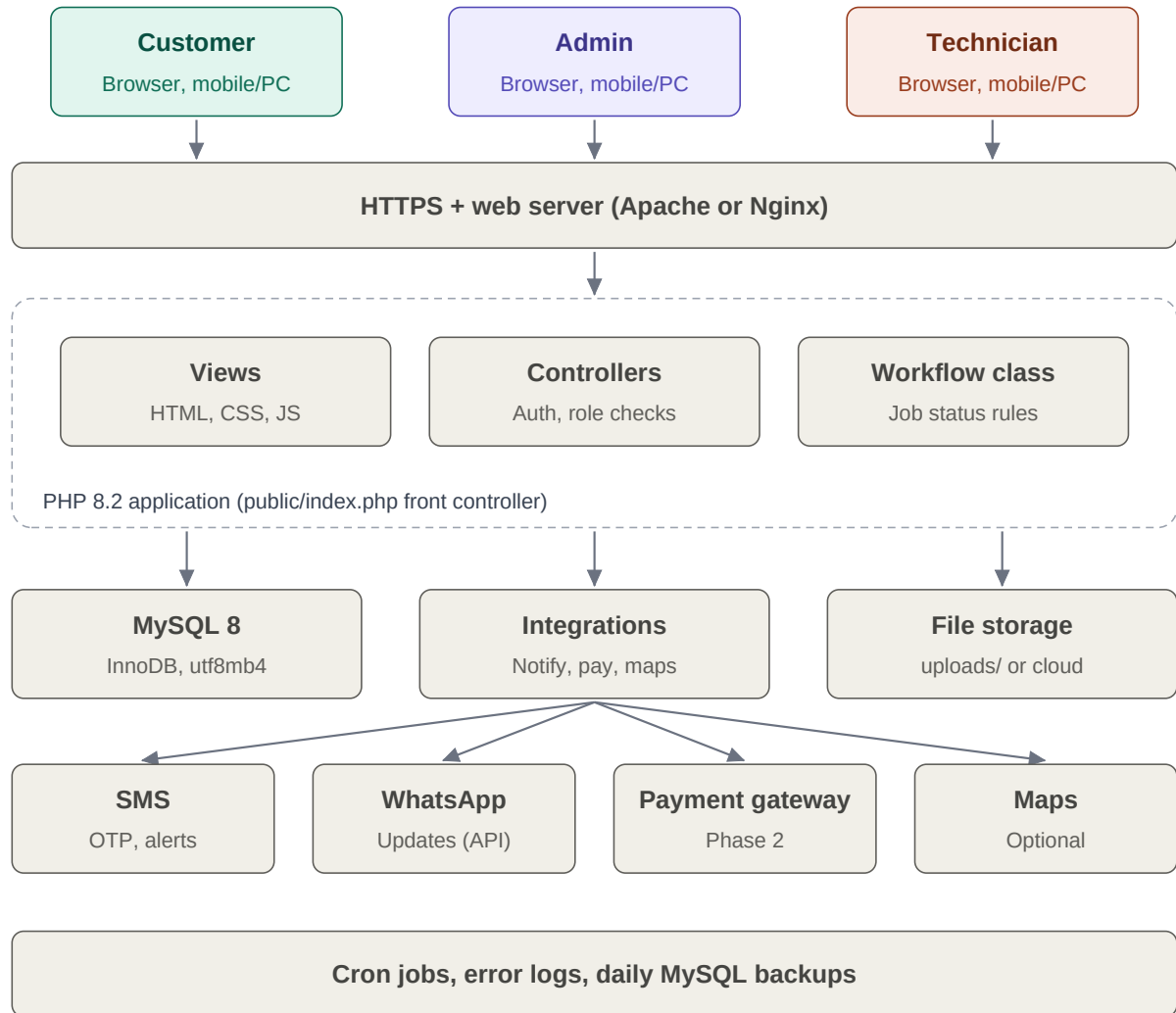
Use A for the hero, B for services and proof, D for About, checklist and process, and E as the mobile behaviour. Choose C instead of A if the client sells many different services.

6. System architecture

The system is shown from four angles: deployment, PHP layers, the service lifecycle, and (in section 7) the database.

6.1 Deployment level

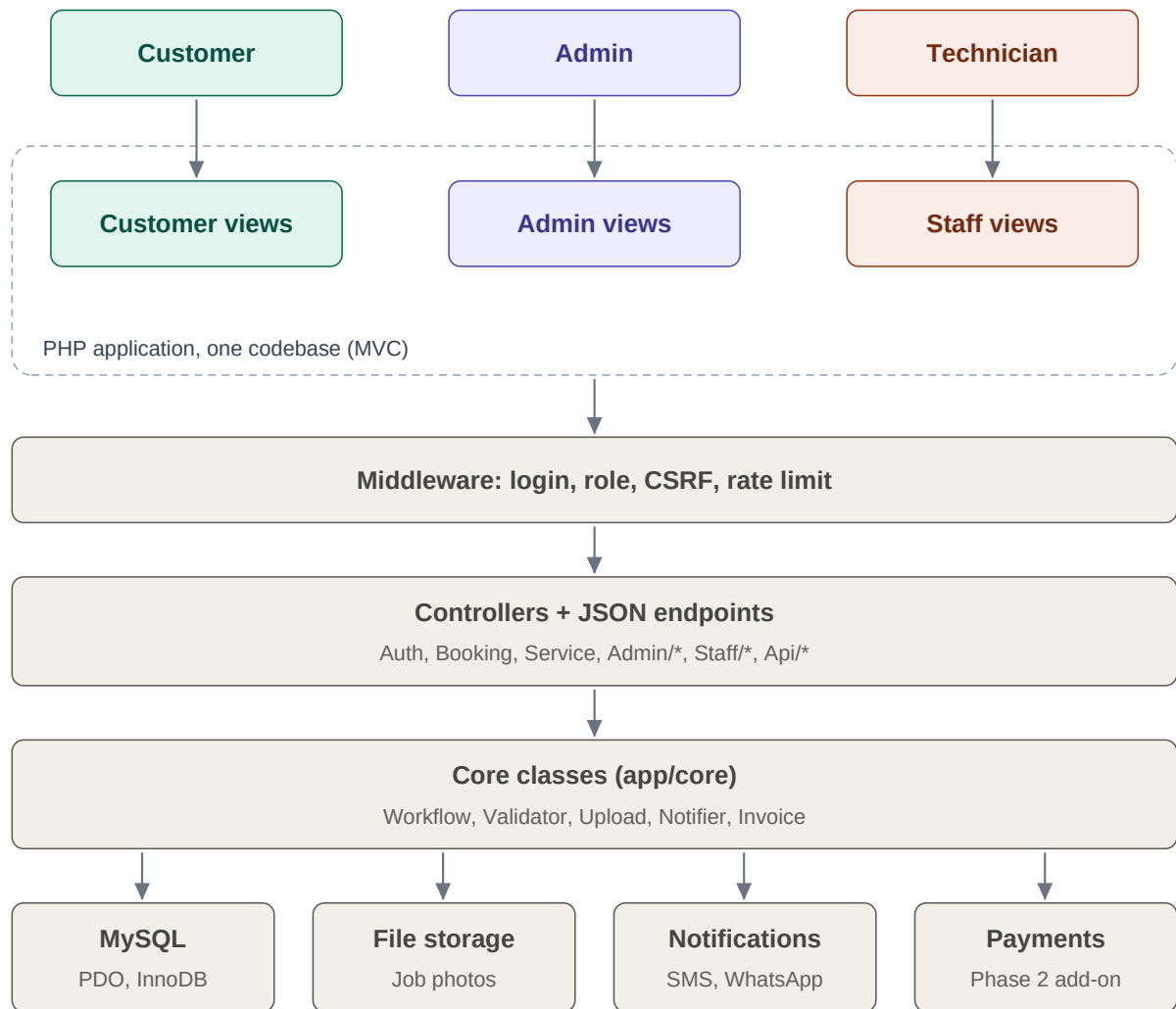
All three users reach one PHP application through HTTPS. Inside it are views, controllers and the workflow class. Data is stored in MySQL, photos in the uploads folder or cloud storage, and SMS, WhatsApp, payments and maps go through an integrations layer.



- Payment gateway and maps are phase 2 / optional. They are not in the scope document.
- Cron jobs send booking reminders and run a daily mysqldump backup. Monitoring and error logs are needed in production even if the document does not list them.

6.2 Application layers (PHP MVC)

Requests enter public/index.php, pass through middleware (login, role, CSRF, rate limit), reach a controller, use core classes for the business rules, and read or write MySQL.



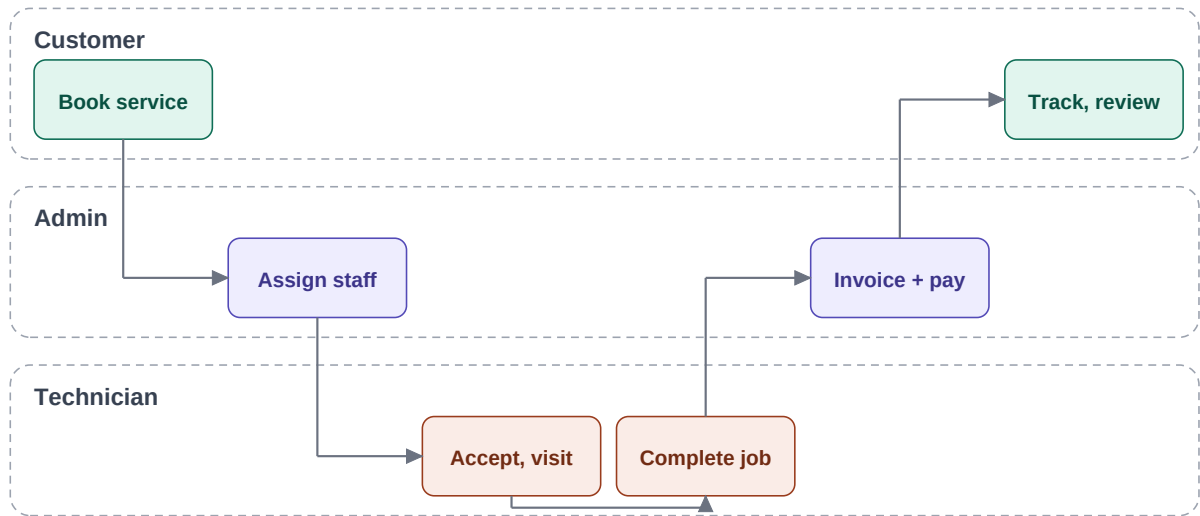
Top-level folder structure

```
service-platform/
|-- public/
|-- app/
|-- config/
|-- database/
|-- cron/
|-- storage/
|-- tests/ docs/
`-- .env composer.json

# web root: index.php, assets, uploads
# controllers, models, views, middleware, core
# app, database, mail, sms, routes
# schema.sql, seed.sql, migrations
# reminders, backup, cleanup
# logs, cache
```

6.3 Service lifecycle (who does what)

The lifecycle from the scope document (Enquiry, Assignment, Acceptance, Service, Completion, Invoice/Review) split across the three roles. The Accept step is an assumption.



Status	Who changes it	Next status
new	Customer submits the booking	assigned
assigned	Admin assigns a technician	accepted
accepted	Technician accepts the job	in_progress
in_progress	Technician marks on the way / started	completed
completed	Technician adds summary and photos	invoiced
invoiced	Admin records payment and receipt	reviewed
reviewed	Customer leaves a rating (module must be added)	closed

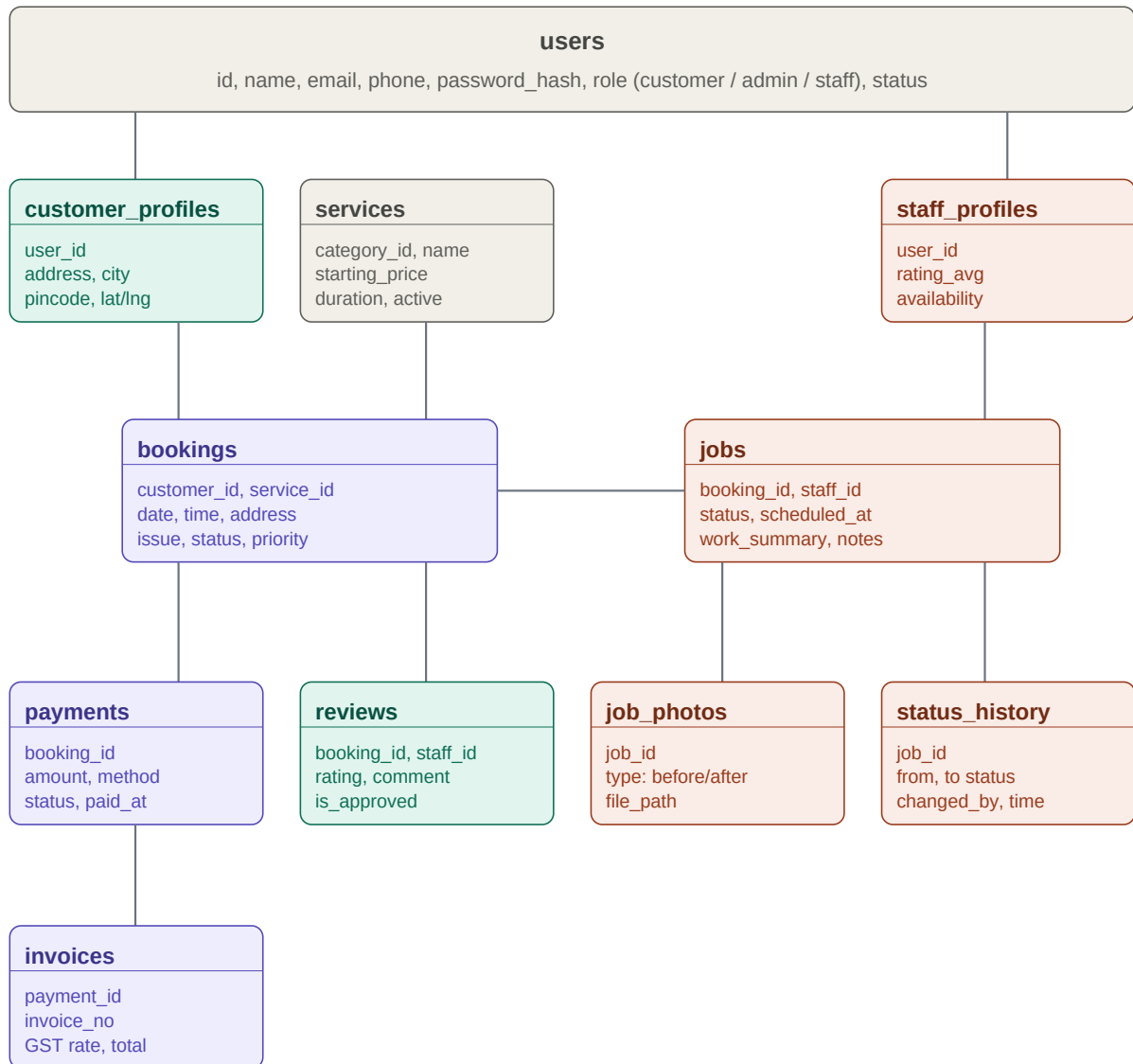
The document has no cancelled, rescheduled or refunded status. Real use will need them; the bookings table below already allows 'cancelled'.

7. MySQL database

MySQL 8 with InnoDB tables, utf8mb4 text and foreign keys. One **users** table holds every login, and the role column separates customers, admins and technicians. All queries go through PDO prepared statements.

7.1 Core relations

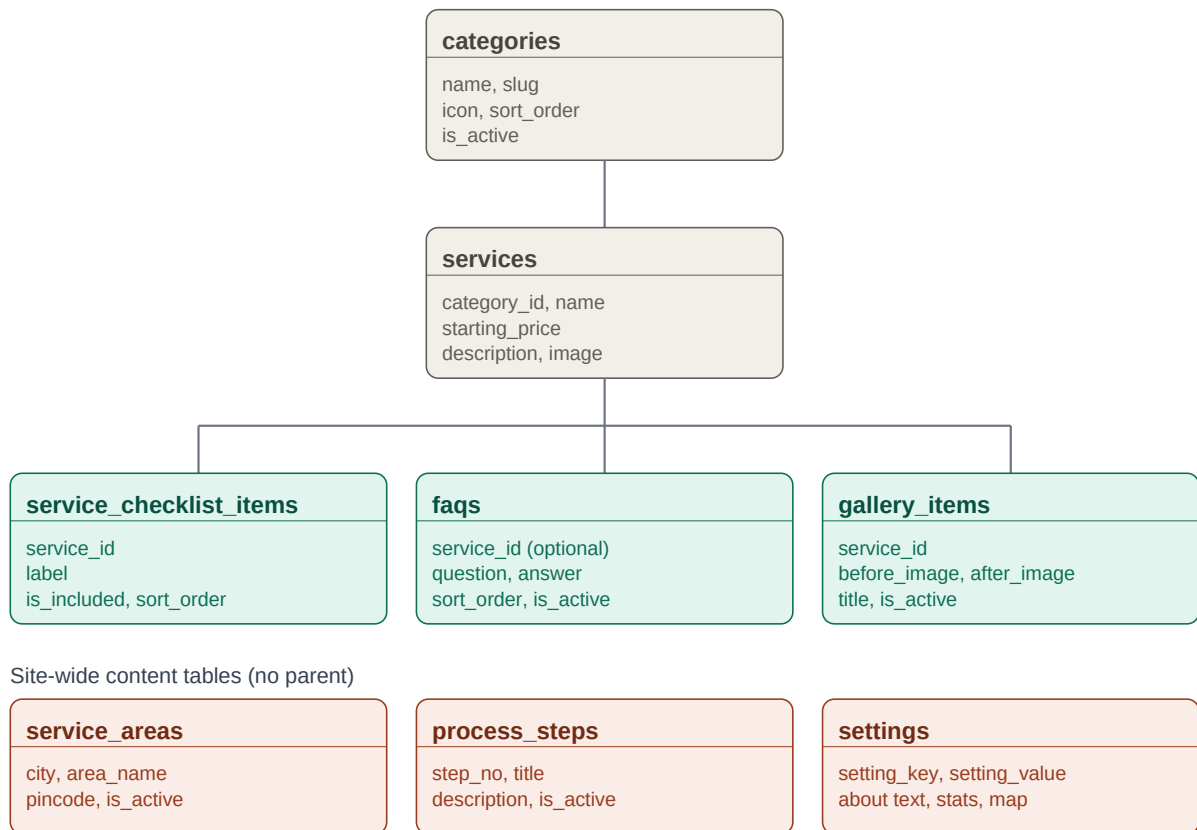
Bookings connect customers to services, and each booking gets one job that links it to a technician. Simple lines show relations (no crow's-foot cardinality).



- One booking has one job; reassigning a technician is recorded in status_history.
- One booking can have several payments, and each payment has one invoice.
- job_photos holds before/after images; a customer's issue photos are stored separately in booking_attachments.

7.2 Website content tables

These tables power the FAQ, gallery, checklist, process, service-area and About sections, and let the admin edit them without a developer.



7.3 All tables

Table	Key columns	Purpose
users	role, name, email, phone, password_hash, status	Every login
customer_profiles	user_id, address, city, pincode, lat/lng	Customer details
staff_profiles	user_id, rating_avg, availability_note	Technician details
staff_skills	staff_id, category_id	Which service types a technician handles
staff_availability	staff_id, weekday, start_time, end_time	Working slots
categories, services	name, slug, starting_price, is_active	Service catalogue
service_checklist_items	service_id, label, is_included	Included / not-included list
service_areas	city, area_name, pincode	Where the business works
bookings	booking_no, customer_id, service_id, status, priority	Enquiries and bookings
booking_attachments	booking_id, file_path, mime, size	Customer issue photos
jobs	booking_id, staff_id, status, work_summary	Assigned work
job_photos	job_id, type, file_path	Before / after images
status_history	job_id, from_status, to_status, changed_by	Audit trail
payments, invoices	amount, status, invoice_no, gst_rate, total	Money records
staff_earnings	job_id, staff_id, amount, month	Technician earnings
reviews	booking_id, rating, comment, is_approved	Customer reviews
faqs, gallery_items, process_steps	question/answer, before/after image, step_no	Website content

Table	Key columns	Purpose
settings	setting_key, setting_value	Business profile, About text, stats, map, alerts
notifications	channel, to_address, template, status	Outgoing message log
password_resets, login_attempts	token_hash, expires_at, ip	Account security
consents	user_id, purpose, granted_at, ip	DPDP consent record

Design notes

- Use foreign keys and indexes on status, staff_id, customer_id and dates so lists and reports stay fast.
- Store money as DECIMAL, never FLOAT. Store times in one timezone (IST) and be consistent.
- Keep uploaded files outside the database; store only the relative path.
- Take a daily mysqldump backup and test a restore before launch.

7.4 SQL schema excerpt (core tables)

This is the starting point for database/schema.sql. Remaining tables follow the same pattern.

```
-- MySQL 8, InnoDB, utf8mb4. Excerpt: core tables. Run each with ENGINE=InnoDB DEFAULT CHARSET=utf8mb4.
CREATE TABLE users (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  role ENUM('customer','admin','staff') NOT NULL,
  name VARCHAR(120) NOT NULL,
  email VARCHAR(160) NULL UNIQUE,
  phone VARCHAR(15) NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  status ENUM('active','disabled') NOT NULL DEFAULT 'active',
  must_change_password TINYINT(1) NOT NULL DEFAULT 0,
  last_login_at DATETIME NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

CREATE TABLE categories (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL, slug VARCHAR(120) NOT NULL UNIQUE,
  description TEXT NULL, icon VARCHAR(60) NULL,
  sort_order INT NOT NULL DEFAULT 0, is_active TINYINT(1) NOT NULL DEFAULT 1
);

CREATE TABLE services (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  category_id INT UNSIGNED NOT NULL,
  name VARCHAR(140) NOT NULL, slug VARCHAR(160) NOT NULL UNIQUE,
  description TEXT NULL, starting_price DECIMAL(10,2) NOT NULL,
  duration_minutes SMALLINT UNSIGNED NULL, image VARCHAR(255) NULL,
  is_active TINYINT(1) NOT NULL DEFAULT 1,
  FOREIGN KEY (category_id) REFERENCES categories(id)
);

CREATE TABLE bookings (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  booking_no VARCHAR(20) NOT NULL UNIQUE,
  customer_id INT UNSIGNED NULL, service_id INT UNSIGNED NOT NULL,
  name VARCHAR(120) NOT NULL, phone VARCHAR(15) NOT NULL, email VARCHAR(160) NULL,
  address VARCHAR(255) NOT NULL, pincode VARCHAR(10) NULL,
  preferred_date DATE NULL, preferred_time VARCHAR(30) NULL, issue_details TEXT NULL,
  status ENUM('new','assigned','accepted','in_progress','completed','invoiced','reviewed','cancelled')
  NOT NULL DEFAULT 'new',
  priority ENUM('low','normal','high','urgent') NOT NULL DEFAULT 'normal',
  admin_notes TEXT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  INDEX (status, created_at), INDEX (customer_id),
  FOREIGN KEY (customer_id) REFERENCES users(id), FOREIGN KEY (service_id) REFERENCES services(id)
);

CREATE TABLE jobs (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  booking_id INT UNSIGNED NOT NULL UNIQUE, staff_id INT UNSIGNED NOT NULL,
  status ENUM('assigned','accepted','in_progress','completed') NOT NULL DEFAULT 'assigned',
  scheduled_at DATETIME NULL, accepted_at DATETIME NULL,
  started_at DATETIME NULL, completed_at DATETIME NULL,
  work_summary TEXT NULL, final_notes TEXT NULL,
  INDEX (staff_id, status),
  FOREIGN KEY (booking_id) REFERENCES bookings(id), FOREIGN KEY (staff_id) REFERENCES users(id)
);

CREATE TABLE job_photos (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, job_id INT UNSIGNED NOT NULL,
  type ENUM('before','after') NOT NULL, file_path VARCHAR(255) NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (job_id) REFERENCES jobs(id) ON DELETE CASCADE
);

CREATE TABLE payments (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, booking_id INT UNSIGNED NOT NULL,
  amount DECIMAL(10,2) NOT NULL, method ENUM('cash','upi','card','bank') NULL,
  status ENUM('pending','partial','paid','refunded') NOT NULL DEFAULT 'pending',
  paid_at DATETIME NULL, recorded_by INT UNSIGNED NULL,
  FOREIGN KEY (booking_id) REFERENCES bookings(id)
);

CREATE TABLE invoices (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, payment_id INT UNSIGNED NOT NULL,
  invoice_no VARCHAR(30) NOT NULL UNIQUE, subtotal DECIMAL(10,2) NOT NULL,
  gst_rate DECIMAL(4,2) NOT NULL DEFAULT 18.00, gst_amount DECIMAL(10,2) NOT NULL,
  total DECIMAL(10,2) NOT NULL, issued_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (payment_id) REFERENCES payments(id)
);
```

```

CREATE TABLE reviews (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, booking_id INT UNSIGNED NOT NULL UNIQUE,
  customer_id INT UNSIGNED NOT NULL, staff_id INT UNSIGNED NULL,
  rating TINYINT UNSIGNED NOT NULL, comment TEXT NULL,
  is_approved TINYINT(1) NOT NULL DEFAULT 0,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (booking_id) REFERENCES bookings(id)
);

CREATE TABLE gallery_items (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, service_id INT UNSIGNED NULL,
  title VARCHAR(140) NOT NULL, before_image VARCHAR(255) NOT NULL, after_image VARCHAR(255) NOT NULL,
  sort_order INT NOT NULL DEFAULT 0, is_active TINYINT(1) NOT NULL DEFAULT 1
);

CREATE TABLE faqs (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, service_id INT UNSIGNED NULL,
  question VARCHAR(255) NOT NULL, answer TEXT NOT NULL,
  sort_order INT NOT NULL DEFAULT 0, is_active TINYINT(1) NOT NULL DEFAULT 1
);

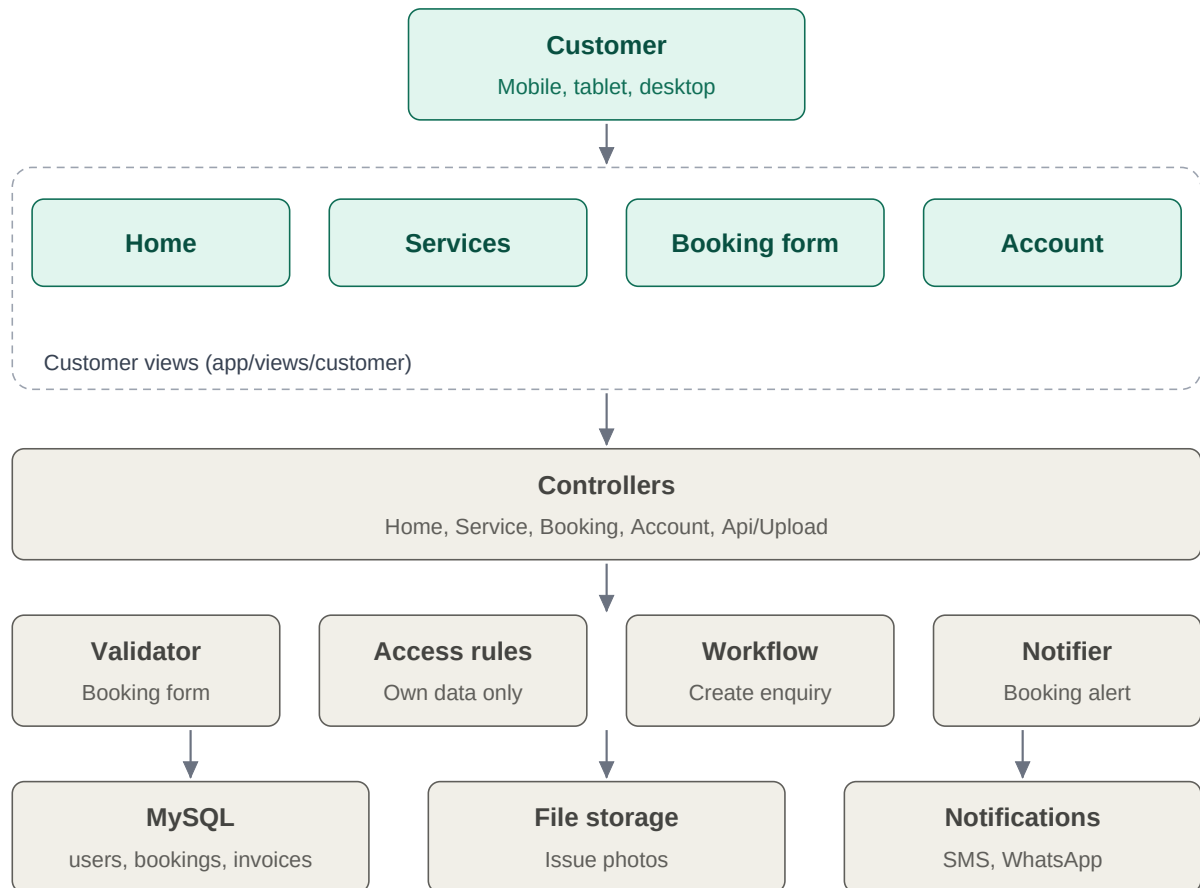
```

8. Customer

The customer browses services, checks the checklist and FAQ, fills the booking form, then follows status and invoices in the account.

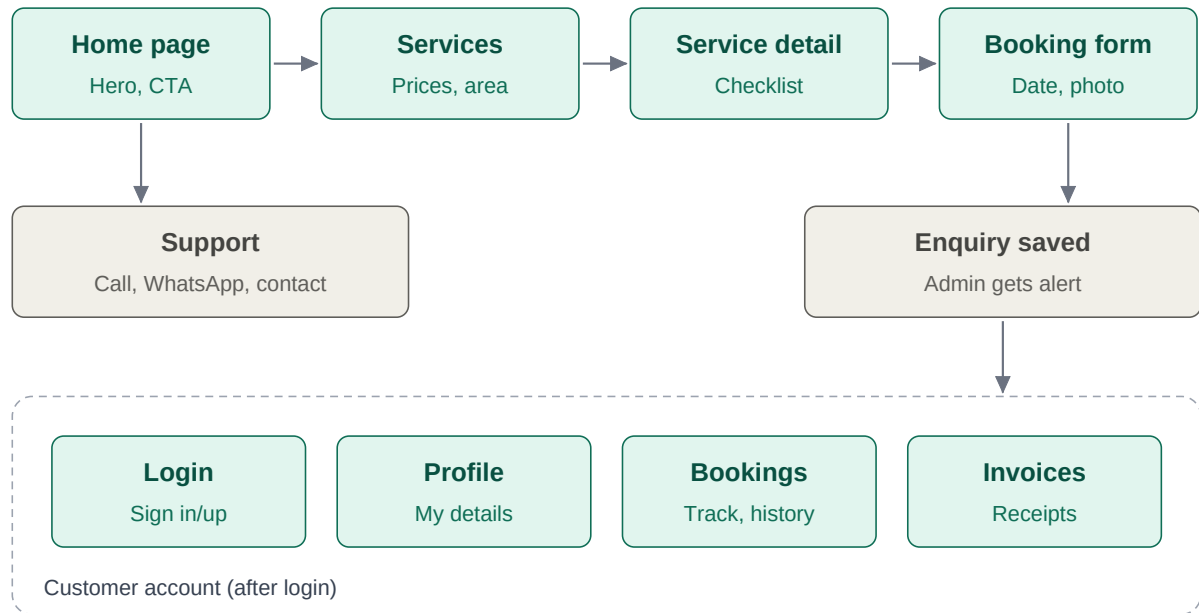
Architecture

Customer views to controllers, core classes and data. A customer sees only their own records.



User flow

From the home page to the account area.



Folder structure

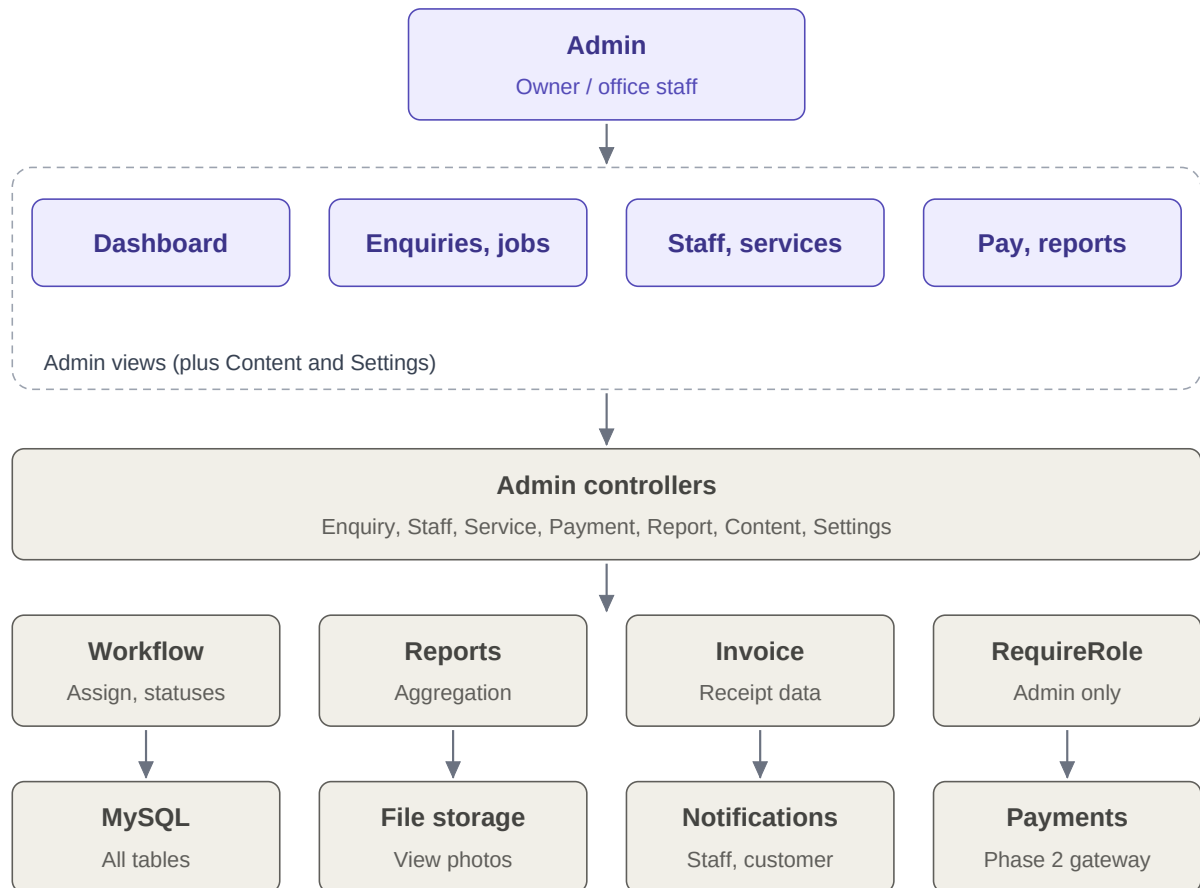
```
app/
|-- controllers/
|   |-- HomeController.php           # home page
|   |-- PageController.php          # about, faq, gallery, contact
|   |-- ServiceController.php        # list + detail
|   |-- BookingController.php        # enquiry / booking form
|   |-- AccountController.php        # profile, bookings, invoices
|   |-- Api/
|       |-- UploadController.php     # issue photo (JSON)
|       |-- ServiceAreaController.php # pincode check (JSON)
|-- models/   Service.php Category.php Booking.php Review.php Faq.php GalleryItem.php
|-- views/
|   |-- layouts/customer.php         # header, footer, sticky call bar
|   |-- customer/
|       |-- home.php about.php faq.php gallery.php contact.php
|       |-- services/
|           |-- index.php             # categories + services
|           |-- show.php              # price, checklist, FAQ, area
|       |-- book.php book-success.php
|       |-- account/
|           |-- index.php profile.php
|           |-- bookings.php booking-detail.php # status timeline
|           |-- invoices.php invoice-detail.php
|   |-- partials/
|       |-- hero.php why-choose-us.php process.php checklist.php
|       |-- before-after.php reviews.php faq-accordion.php
|       |-- map.php enquiry-form.php mobile-bar.php
public/assets/
|-- css/   base.css components.css customer.css
|-- js/    booking.js before-after.js faq.js review-slider.js service-area.js
```

9. Admin

The admin has eight sidebar areas: dashboard, enquiries and jobs, staff, services, payments, reports, website content and settings.

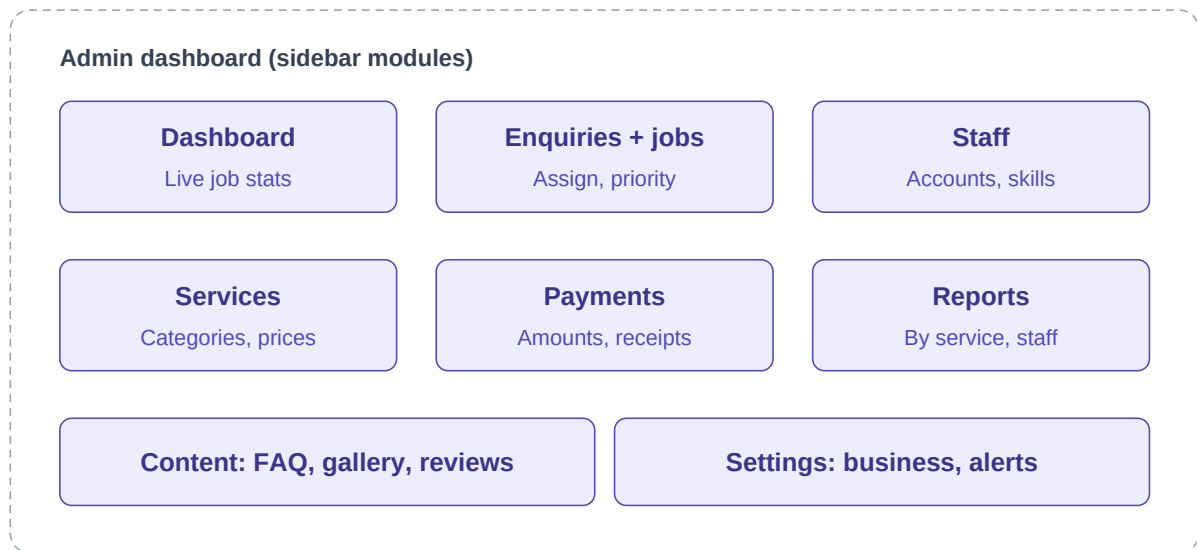
Architecture

The admin can reach all data, so RequireRole allows only the admin role.



Modules and daily flow

Sidebar modules and the everyday routine.



Daily flow



Folder structure

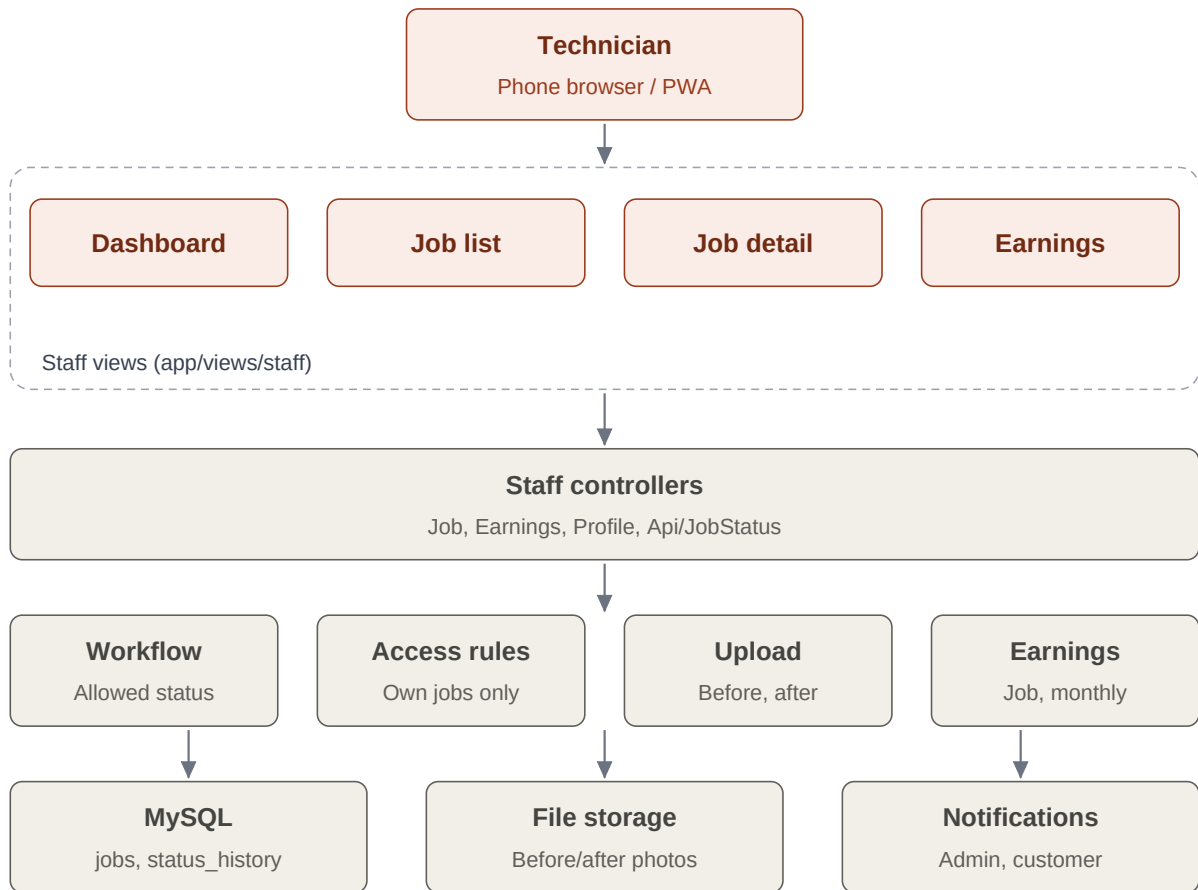
```
app/
|-- controllers/Admin/
|   |-- DashboardController.php
|   |-- EnquiryController.php           # list, detail, notes, priority
|   |-- BookingController.php          # scheduled, active, completed, assign
|   |-- StaffController.php
|   |-- ServiceController.php          # services + categories
|   |-- PaymentController.php          # payments + invoices
|   |-- ReportController.php
|   |-- ContentController.php          # FAQs, gallery, checklists, reviews, areas, steps
|   |-- SettingsController.php
|-- views/
|   |-- layouts/admin.php              # sidebar + top bar
|   |-- admin/
|   |   |-- dashboard.php
|   |   |-- enquiries/ index.php show.php
|   |   |-- bookings/ index.php show.php calendar.php
|   |   |-- staff/ index.php create.php show.php
|   |   |-- services/ index.php form.php categories.php
|   |   |-- payments/ index.php show.php
|   |   |-- reports/ index.php services.php staff.php bookings.php
|   |   |-- content/ faqs.php gallery.php checklists.php reviews.php areas.php steps.php pages.php
|   |   |-- settings/ index.php business.php notifications.php
|   |-- partials/ stat-card.php data-table.php assign-modal.php date-filter.php
public/assets/
|-- css/ admin.css
|-- js/ admin.js charts.js assign-modal.js
```

10. Technician

The technician sees assigned jobs, updates status, uploads before/after photos with notes, and checks earnings. The Accept / decline step is an assumption.

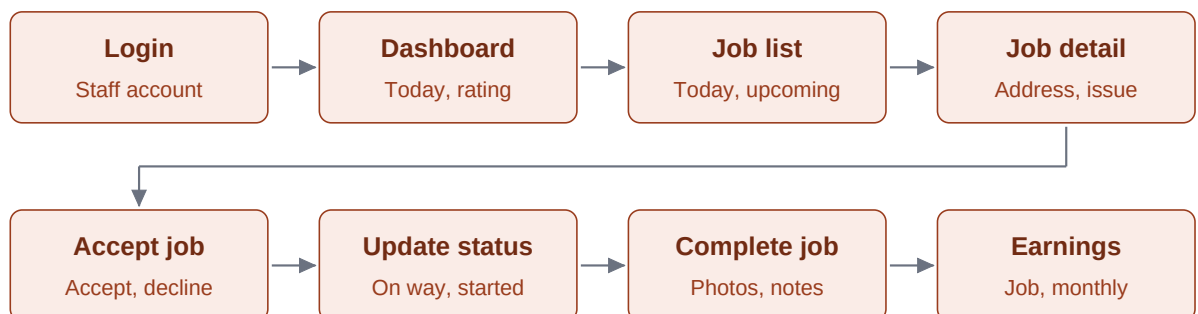
Architecture

A technician sees only their own assigned jobs; the SQL query enforces it.



Job flow

From login to earnings.



Folder structure

```

app/
|-- controllers/
|   |-- Staff/
|   |   |-- DashboardController.php
|   |   |-- JobController.php           # list, detail, complete
|   |   |-- EarningsController.php
|   |   |-- ProfileController.php
|   |   |-- Api/JobStatusController.php # status updates (JSON)
|-- models/ Job.php JobPhoto.php StatusHistory.php StaffProfile.php StaffEarning.php
|-- views/
|   |-- layouts/staff.php               # mobile-first, bottom nav
|   |-- staff/
|   |   |-- dashboard.php
|   |   |-- jobs/ index.php show.php complete.php
|   |   |-- earnings.php history.php profile.php
|   |-- partials/ job-card.php status-buttons.php photo-uploader.php
public/assets/
|-- css/ staff.css
|-- js/ staff-jobs.js photo-upload.js

```

11. Shared code

Shared piece	Where it lives	Used by
Login and session	AuthController, core/Auth.php	All three
Role check	middleware/RequireRole.php	All three
CSRF and rate limit	middleware/VerifyCsrf.php, RateLimit.php	All three
Job status rules	core/Workflow.php	Admin, Technician, Customer (to show status)
Database access	core/Database.php (PDO), models/	All three
Photo upload	core/Upload.php, Api/UploadController	Customer (issue photo), Technician (before/after)
Notifications	core/Notifier.php	Admin alerts, customer status updates
Invoices	core/InvoiceBuilder.php	Admin creates, customer views

12. Feature matrix: which feature is used where

Main = this role creates or edits the data. **View** = read-only, or receives a notification. - = not used by this role.

Feature	Customer	Admin	Tech	Where it lives	MySQL tables
Login / signup	Main	Main	Main	AuthController, views/auth	users
Role-based access	View	View	View	RequireRole.php, core/Auth.php	users.role
Service catalogue (categories, prices, area)	View	Main	View	ServiceController, Admin/ServiceController	categories, services
Service checklist (included / not)	View	Main	View	partials/checklist.php, Admin/ContentController	service_checklist_items
Booking / enquiry form	Main	View	-	BookingController, partials/enquiry-form.php	bookings
Service-area / pincode check	Main	Main (manage)	-	Api/ServiceAreaController	service_areas
Photo / file upload	Main (issue)	View	Main (before/after)	core/Upload.php, Api/UploadController	booking_attachments, job_photos
Enquiry management (status, notes, priority)	-	Main	-	Admin/EnquiryController	bookings
Assign / reassign staff	-	Main	View	Admin/BookingController	jobs
Job list and details	View (status)	Main	Main	Staff/JobController	jobs
Job status updates	View (timeline)	View	Main	Api/JobStatusController, core/Workflow.php	jobs, status_history
Job completion (summary, photos, notes)	View	View	Main	Staff/JobController	jobs, job_photos
Payment record	View (status)	Main	-	Admin/PaymentController	payments
Invoice / receipt	View	Main	-	core/InvoiceBuilder.php	invoices
Review / rating	Main	Main (approve)	View (own)	AccountController, Admin/ContentController	reviews
Earnings	-	View (report)	Main	Staff/EarningsController	staff_earnings
Staff management	-	Main	View (own profile)	Admin/StaffController	staff_profiles, staff_skills
Profile management	Main	Main	Main	AccountController, Staff/ProfileController	users, profiles
Booking / service history	Main	Main (all)	Main (own)	AccountController, Staff/JobController	bookings, jobs
Dashboard stats	-	Main	View (own)	Admin/DashboardController	bookings, jobs, payments
Reports and date filters	-	Main	-	Admin/ReportController, core/Report.php	all tables
Settings	-	Main	-	Admin/SettingsController	settings
Notifications (SMS / WhatsApp / email)	View	View	View	core/Notifier.php	notifications
Call / WhatsApp support	Main	-	-	partials/mobile-bar.php, contact.php	none
About page and stats	View	Main	-	PageController, Admin/ContentController	settings
Before/after gallery	View	Main	-	partials/before-after.php	gallery_items

Feature	Customer	Admin	Tech	Where it lives	MySQL tables
Process steps	View	Main	-	partials/process.php	process_steps
FAQ	View	Main	-	partials/faq-accordion.php	faqs
Location map	View	Main	-	partials/map.php	settings, service_areas
Responsive UI	Main	Main	Main	assets/css, views/layouts	none

Where features are used during one job

1. The customer fills the booking form and uploads a photo (form, upload, validation).
2. The admin gets a notification, reviews the enquiry, sets priority and assigns a technician.
3. The technician accepts the job (assumption; not defined in the document).
4. The technician updates status (on the way, started); the customer sees the timeline.
5. The technician finishes and adds a summary and before/after photos.
6. The admin records payment and creates the receipt; the customer views the invoice.
7. The customer leaves a rating (module not in the document).
8. The technician's earnings update and the data appears in the admin reports.

Shared vs role-specific

- **Shared:** login, role check, profile, responsive UI, photo upload, notifications, and the job status rules in core/Workflow.php.
- **Customer only:** booking form, call/WhatsApp support.
- **Admin only:** enquiry management, assignment, staff and service management, payments, reports, website content, settings.
- **Technician only:** job status updates, completion form, earnings.

Still missing from the scope document

- Online payment gateway
- Review / rating module
- Cancel, reschedule and refund
- A fixed choice of notification channel
- GST-compliant invoice

13. Full folder structure (detailed)

The complete proposed tree, in four parts. Names such as Admin/ and Staff/ are sub-folders of controllers; lines starting with # are notes. Only public/ should be reachable from the web.

13.1 Root, public files, config, database, cron, docs and tests

```
service-platform/
|-- .env                                     # DB credentials, mail, SMS keys (never public)
|-- .env.example
|-- .gitignore
|-- composer.json                         # phunit, phpdotenv, phpmailer (optional)
|-- README.md
|-- public/                               # the only web-accessible folder
|   |-- index.php                         # front controller
|   |-- .htaccess                         # clean URLs, security headers
|   |-- assets/
|   |   |-- css/ base.css components.css customer.css admin.css staff.css print.css
|   |   |-- js/ app.js booking.js before-after.js faq.js review-slider.js service-area.js
|   |   |-- js/ admin.js charts.js assign-modal.js staff-jobs.js photo-upload.js
|   |   |-- img/ logo.svg favicon.ico hero/ icons/
|   |-- uploads/
|   |   |-- .htaccess                     # blocks PHP execution
|   |   |-- issues/                       # customer issue photos
|   |   |-- jobs/                         # before / after photos
|   |   |-- gallery/                     # public before-after images
|-- config/
|   |-- app.php database.php mail.php sms.php
|   |-- routes.php                       # URL -> controller map
|-- database/
|   |-- schema.sql                       # all MySQL tables
|   |-- seed.sql                         # demo categories, services, first admin
|   |-- migrations/                     # numbered .sql files
|-- cron/
|   |-- send-reminders.php               # booking reminders
|   |-- daily-backup.php                 # mysqldump
|   |-- cleanup-uploads.php
|-- storage/
|   |-- logs/ cache/
|-- docs/
|   |-- requirements.md db-schema.md api-reference.md deployment.md handover-walkthrough.md
|-- tests/
|   |-- Unit/ WorkflowTest.php ValidatorTest.php InvoiceTest.php RoleAccessTest.php
|   |-- Feature/ AuthTest.php BookingFlowTest.php AssignmentTest.php PermissionTest.php
|   |-- Browser/ responsive-checklist.md # mobile, tablet, desktop
```

13.2 app/: core classes, middleware, models and controllers

```
app/
|-- core/
|   |-- Router.php Request.php Response.php View.php
|   |-- Database.php                     # PDO, prepared statements
|   |-- Env.php Logger.php
|   |-- Auth.php Csrf.php Validator.php
|   |-- Upload.php                       # type, size, rename checks
|   |-- Workflow.php                     # Enquiry > Assignment > Acceptance > Service > Completion > Invoice
|   |-- Notifier.php                     # SMS, email, WhatsApp
|   |-- InvoiceBuilder.php                 # receipt and GST invoice
|   |-- Report.php                       # aggregation queries
|-- middleware/
|   |-- RequireLogin.php RequireRole.php VerifyCsrf.php RateLimit.php
|-- models/
|   |-- User.php CustomerProfile.php StaffProfile.php StaffEarning.php
|   |-- Category.php Service.php ServiceChecklistItem.php ServiceArea.php
|   |-- Booking.php BookingAttachment.php Job.php JobPhoto.php StatusHistory.php
|   |-- Payment.php Invoice.php Review.php
|   |-- Faq.php GalleryItem.php ProcessStep.php Setting.php
|   |-- Notification.php PasswordReset.php LoginAttempt.php Consent.php
|-- controllers/
|   |-- AuthController.php HomeController.php PageController.php
|   |-- ServiceController.php BookingController.php AccountController.php
|   |-- Admin/ Dashboard Enquiry Booking Staff Service Payment Report Content Settings
|   |-- Staff/ Dashboard Job Earnings Profile
|   |-- Api/ Upload ServiceArea JobStatus Notification
```

13.3 app/views: every template and partial

```
app/views/
|-- layouts/ customer.php admin.php staff.php auth.php
|-- auth/ login.php signup.php forgot-password.php reset-password.php change-password.php
|-- customer/
|   |-- home.php about.php faq.php gallery.php contact.php book.php book-success.php
```

```

| |-- services/ index.php show.php
| |-- account/ index.php profile.php bookings.php booking-detail.php invoices.php invoice-detail.php
|-- admin/
| |-- dashboard.php
| |-- enquiries/ index.php show.php
| |-- bookings/ index.php show.php calendar.php
| |-- staff/ index.php create.php show.php
| |-- services/ index.php form.php categories.php
| |-- payments/ index.php show.php
| |-- reports/ index.php services.php staff.php bookings.php
| |-- content/ faqs.php gallery.php checklists.php reviews.php areas.php steps.php pages.php
| |-- settings/ index.php business.php notifications.php
|-- staff/
| |-- dashboard.php
| |-- jobs/ index.php show.php complete.php
| |-- earnings.php history.php profile.php
|-- partials/
| |-- navbar.php footer.php mobile-bar.php flash.php pagination.php
| |-- hero.php why-choose-us.php process.php checklist.php before-after.php
| |-- reviews.php faq-accordion.php map.php enquiry-form.php
| |-- stat-card.php data-table.php assign-modal.php date-filter.php
| |-- job-card.php status-buttons.php photo-uploader.php empty-state.php
|-- emails/ booking-confirmation.php job-assigned.php invoice.php password-reset.php

```

Which folders each role touches

Folder	Customer	Admin	Technician
views/customer, controllers (root)	Main	-	-
views/admin, controllers/Admin	-	Main	-
views/staff, controllers/Staff	-	-	Main
views/auth, AuthController	Yes	Yes	Yes
controllers/Api	Some	Some	Some
views/partial (site sections)	Main	-	-
views/partial (tables, modals)	-	Main	-
core/Workflow, Auth, Upload, Notifier	Yes	Yes	Yes
core/InvoiceBuilder, Report	View	Main	-
database/schema.sql	Yes	Yes	Yes

14. Security and deployment checklist

Plain PHP does not protect you by default, so these controls must be built in from the start.

Risk	Control
SQL injection	PDO prepared statements for every query; never build SQL from user input
Weak passwords / hijacked sessions	password_hash and password_verify; session_regenerate_id on login; secure, HttpOnly, SameSite cookies
CSRF	CSRF token in every form and every fetch POST (VerifyCsrf middleware)
XSS	htmlspecialchars on all output; a Content-Security-Policy header
Broken access control	RequireRole on every admin/staff page and JSON endpoint; technician queries filtered by staff_id
Dangerous uploads	Check MIME and extension, size limit, random file names, store outside code paths, block PHP execution in uploads/
Brute force	Rate limit login and OTP attempts; lock after repeated failures
Exposed secrets	.env outside the web root; never commit it; separate keys for test and production
Data loss	Daily mysqldump, off-server copy, and a tested restore
Privacy (DPDP)	Consent record, privacy notice, data deletion process, access logs

Hosting and go-live

- PHP 8.2+, MySQL 8 or MariaDB 10.6+, extensions pdo_mysql, mbstring, fileinfo, gd, curl; HTTPS with auto-renewing certificate.
- Point the web root at public/. Keep app/, config/ and .env outside the public folder so nobody can open them by URL.
- Set display_errors off in production and write errors to storage/logs.
- Configure cron for reminders, backups and upload cleanup.
- Test on real phones (Android and iPhone) and on a slow connection before handover.

15. Next steps

- Decide whether this is a tool for one business or a product for many, and whether technicians are employees or contractors.
- Choose plain PHP or Laravel, and confirm the hosting plan meets the requirements above.
- Split the missing-features list into phase 1 (launch) and phase 2 (payment gateway, GST invoice, WhatsApp automation, reviews).
- Pick a home page layout from section 5 and collect real photos, reviews and FAQ content from the client.
- Confirm exact job statuses, including accept/decline, cancel, reschedule and refund.
- Ask the vendor for price, source-code rights, hosting ownership and a support agreement in writing.
- A scaffold script that creates all folders and empty files, plus a complete schema.sql, can be generated on request.